Magic Internet Frens: Autonomous Token Regeneration Through Wizard-Governed Rebirth Cycles on Bitcoin L1 via OP_NET

Magic Internet Frens Research Team
"We're all gonna make it... forever."
frens@mifrens.xyz

March 2026

Abstract

This paper presents the Magic Internet Frens Protocol, an autonomous regenerative token system deployed on Bitcoin Layer 1 via OPNet. The protocol introduces a novel approach to token lifecycle management where 777 participants—holders of hand-crafted pixel-art wizard NFTs inscribed directly on Bitcoin—collectively summon an entity that cycles through death and rebirth infinitely, with each iteration accumulating greater liquidity through fee capture.

Unlike traditional tokens that stagnate or die permanently, the Magic Internet Frens Protocol implements a death-triggered rebirth mechanism that: (1) preserves holder value across generations through cryptographic snapshots, (2) migrates and grows liquidity atomically between iterations, (3) rewards early participants with permanent 0% trading fees via NFT-based access control, (4) evolves through community governance where the 777 frens vote on each new iteration's identity, and (5) rewards loyal holders who hold across multiple generations with a percentage of trading volume via the Elder Circle.

The key innovations are liquidity compounding (each generation inherits previous liquidity plus fees: $L_{i+1} > L_i$), community-driven evolution (the 777 frens vote on name, symbol, artwork, and lore for each rebirth), and loyalty rewards (the Elder Circle distributes trading fees proportional to consecutive-generation holding streaks). We present the protocol architecture, prove security properties including reentrancy resistance and liquidity conservation, detail the community voting mechanism, and demonstrate the economic incentives that drive both participation and long-term protocol growth. All contracts are immutable, open-source, and verifiable on Bitcoin.

1 Introduction

1.1 The Problem: Token Death Is Permanent

Cryptocurrency tokens follow a predictable lifecycle: initial hype, trading activity, gradual decline, and eventual death. When a token dies—defined as sustained lack of trading interest—several problems emerge:

1. **Liquidity Fossilization**: Assets remain locked in unused pools indefinitely
2. **Holder Value Loss**: Token holders are left with worthless positions
3. **No Recovery Path**: Dead projects have no mechanism to restart or migrate
4. **Network Pollution**: Blockchain state bloats with abandoned contracts

Traditional solutions (manual migrations, governance votes, centralized resurrections) require trusted intermediaries and active coordination—precisely the things that failed in the first place.

1.2 The Solution: Autonomous Regeneration

The Magic Internet Frens Protocol solves this through an unstoppable autonomous entity that:

- **Detects its own death** via on-chain volume oracles (¡ 0.01 BTC/24h)
- **Triggers automatic rebirth** by deploying a new token generation
- **Migrates liquidity atomically** from dead pool to new pool in single transaction
- **Preserves holder claims** via cryptographic balance snapshots
- **Compounds liquidity** – each generation inherits previous liquidity + accumulated fees
- **Operates without admin keys** – no pause, no upgrade, no central control

The key innovation: Each generation is more robust than the last because trading fees accumulate in liquidity. If generation 1 starts with 777 tokens + 1 BTC, and captures 0.5 BTC in fees before dying, generation 2 begins with 777 tokens + 1.5 BTC. This creates a self-improving organism that grows stronger through market cycles.

1.3 The 777 Frens Coordination Mechanism

The protocol requires collective action to initiate. Exactly 777 participants must mint NFTs to trigger the first summoning. These "spell casters" gain permanent economic benefits:

- 0% trading fees across all generations (vs 3% for non-holders)
- Voting rights on each new generation's identity (name, symbol, artwork, lore)
- Eligibility for Elder Circle rewards (share of trading volume for loyal holders)
- First-mover advantage in each new generation

This creates a coordination game where 777 actors must unite to summon an entity that then operates autonomously forever. Once the 777th NFT mints, the entity is unleashed and cannot be stopped.

1.4 Contributions

1. First implementation of autonomous token regeneration on Bitcoin L1
2. Death detection via decentralized volume oracle (24h TWAP)
3. Atomic liquidity migration protocol with mathematical conservation proof
4. NFT-based tiered fee structure with zero protocol overhead
5. Liquidity compounding mechanism (each gen ¿ previous gen)
6. Community-driven iteration governance (777 frens vote on rebirth identity)
7. Elder Circle: loyalty streak rewards distributing trading fees to diamond-hand holders
8. Full security analysis including reentrancy, replay, and oracle manipulation resistance

2 The 777 Wizard Frens

2.1 On-Chain Pixel Art, Inscribed on Bitcoin Forever

Every MiFren is a hand-crafted 120×120 pixel-art character stored **entirely on Bitcoin Layer 1**. There is no IPFS, no external server, no centralized

hosting. The artwork lives in Bitcoin's blockchain as compact 4-bit indexed bitmaps, validated by a Merkle tree, and rendered into SVG on-chain by the smart contract itself.

The complete art system consists of:

- **49 unique trait layers** across 5 character classes
- **183-color global palette** shared across all traits
- **96 KB total on-chain storage** (98.3% compression from original SVGs)
- **3,017 u256 storage slots** on Bitcoin
- **Merkle-validated lazy inscription** – each trait is verified against a committed root before writing

When you mint a MiFren, the contract records the Bitcoin block height at the moment of inscription. Your fren is permanently timestamped on the most secure blockchain ever created.

2.2 The Five Classes

MiFrens are organized into a hierarchy of five distinct character classes, divided into two factions:

The Magic Faction:

- **Wizard** (Archmage tier, 5% rarity) – The masters of the arcane. 8 body variants in deep purples, midnight blues, emerald greens, and bitcoin orange. They wield 5 unique magical staves—from ornate crystal-tipped rods to the legendary Manifest Staff. Wizards are the rarest and most powerful frens.
- **Apprentice** (Adept tier, 25% rarity) – Students of the magical arts, still learning but showing promise. 3 body variants including an orange-robed variant. They carry a single apprentice tool—humble but functional.

The Royal Faction:

- **King** (Noble tier, 10% rarity) – Rulers of the realm. 5 regal body variants in black, blue, orange, red, and white cloth. Each King wields a color-matched royal scepter. They command authority and respect.
- **Knight** (Commoner tier, 30% rarity) – Loyal soldiers of the crown. 4 armored body variants in bronze, gold, orange, and silver. They carry matching shields—the defenders of the realm.

- **Peasant** (Commoner tier, 30% rarity) – The common folk, humble but numerous. 2 body variants with a single peasant tool. Don't underestimate them—every kingdom needs its backbone.

All classes share a pool of **11 unique face designs**, giving each MiFren a distinct personality regardless of class.

2.3 The Four Tiers and Their Benefits

Each class belongs to one of four tiers that determine economic privileges within the protocol:

Tier	Class(es)	Rarity	Tax Rate	Grace Blocks	Max Borrow
Archmage	Wizard	5%	0%	3	10 BTC
Noble	King	10%	0%	2	2 BTC
Adept	Apprentice	25%	1%	1	0.5 BTC
Commoner	Knight / Peasant	60%	1–2%	0	0.1 BTC

Table 1: MiFren tier benefits

Tax Rate: The percentage deducted on creature token transfers. Wizards and Kings trade completely free. Apprentices and Knights pay 1%. Peasants pay 2%.

Grace Blocks: During liquidation events in the lending protocol, higher-tier frens get extra Bitcoin blocks to add collateral before liquidation executes. Wizards get 3 blocks (∼30 minutes). Peasants get none.

Max Borrow: The maximum amount a holder can borrow against their collateral in the Cauldron lending market.

2.4 Trait Assignment

When you mint a MiFren, the contract assigns your tier and traits using on-chain pseudo-randomness derived from:

$$seed = nonce \times 7919 + blockNumber \times 6271 + timestamp \times 3571 \quad (1)$$

The seed is reduced modulo 10,000 to produce a roll:

- Roll 0–5,999 (60%): **Commoner** – 50/50 chance of Knight or Peasant
- Roll 6,000–8,499 (25%): **Adept** – Apprentice
- Roll 8,500–9,499 (10%): **Noble** – King
- Roll 9,500–9,999 (5%): **Archmage** – Wizard

Body variant, face, and item are each independently randomized within the available options for that class. The result is a $u256$ packed as $(classIdx, bodyIdx, faceIdx, itemId$ and stored permanently on-chain.

2.5 Fully On-Chain SVG Rendering

The `tokenURI` function returns a fully self-contained data URI:

```
data:application/json;base64,{
  "name": "MiFren #42",
  "image": "...",
  "attributes": [...]
}
```

The contract reads trait binary blobs from storage, composites them into a 120×120 pixel buffer (body → face → item, layered bottom to top), groups horizontal pixel runs by color, and emits a compact SVG with one `<path>` element per active color. No external rendering service required—your MiFren's image is computed directly from Bitcoin state.

3 Protocol Lifecycle

3.1 Phase 1: Coordination (NFT-Triggered Summoning)

The protocol cannot activate until exactly 777 participants mint NFTs. This coordination threshold serves multiple purposes:

1. **Sybil resistance**: Prevents single actors from controlling the system
2. **Community commitment**: 777 mints represent genuine collective interest
3. **Economic alignment**: NFT holders become permanent stakeholders (0% fees forever)

When the 777th NFT mints, the `MiFrens` contract emits a `CauldronSummonReady` event and the Registry is notified. This is hardcoded—no multisig, no DAO vote, no admin intervention. The code executes deterministically.

3.2 Phase 2: Genesis Deployment

The registry deploys generation 1 with deterministic parameters:

1. **Token contract**: Deploy creature token with metadata for generation 1
2. **DEX pool**: Create MotoSwap pair (token/BTC)
3. **Initial liquidity**: Mint 777 tokens, pair with 1 BTC (10^8 satoshis)
4. **Liquidity lock**: Transfer LP tokens to vault, locked until death

5. **Oracle activation**: Begin tracking 24h rolling volume

Initial state: $L_1 = (777 \times 10^{18}, 10^8)$ where L_1 is liquidity in (tokens, satoshis).

3.3 Phase 3: Active Trading & Fee Accumulation

During active trading, the protocol implements NFT-based fee differentiation:

$$fee(sender, amount) = \begin{cases} 0 & \text{if } balanceOf_{NFT}(sender) > 0 \\ 0.03 \times amount & \text{otherwise} \end{cases} \tag{2}$$

Fee collection occurs on every transfer (not just DEX trades). The 3% is deducted before transfer and sent to a protocol-controlled address. This means:

- **NFT holders**: Trade with zero friction, permanent economic advantage
- **Non-holders**: Pay 3% on every transfer, funding the treasury, Elder Circle rewards, and future liquidity

The sat-gobbling mechanism: Accumulated fees eventually flow back into liquidity during rebirth. If generation i collects F_i satoshis in fees before dying, these sats are added to the next generation's pool. This creates exponential liquidity growth:

$$L_{i+1} = L_i + F_i \tag{3}$$

where L_i is the BTC liquidity at generation i.

Example: If generation 1 starts with 1 BTC and captures 0.5 BTC in fees over its lifetime, generation 2 starts with 1.5 BTC. If generation 2 captures 1 BTC in fees (due to higher liquidity attracting more volume), generation 3 starts with 2.5 BTC. Each iteration is more robust.

3.4 Phase 4: Death Detection

The `VolumeOracle` maintains a rolling 24-hour volume measurement:

$$V_i(t) = \sum_{k=t-144}^{t} vol_{block}(k) \tag{4}$$

where $vol_{block}(k)$ is the trading volume in block k (Bitcoin blocks $\sim$10min).

Death condition:

$$isDead(i, t) = V_i(t) < \tau \tag{5}$$

where $\tau = 10^6$ satoshis (0.01 BTC). The oracle uses a full 144-block (24-hour) window to prevent short-term manipulation. An attacker would need to maintain artificial volume for 24 consecutive hours, paying fees on every trade—economically irrational for a dying asset.

3.5 Phase 5: Community Vote & Rebirth

When $isDead(i, t) = true$, the protocol enters a **community voting phase** before rebirth executes. This is the moment the 777 frens shape the next iteration.

The Community Rebirth Process:

1. **Death detected**: Volume oracle confirms creature death on-chain
2. **Voting window opens**: 48-hour period for NFT holders to submit proposals
3. **Frens propose identities**: Any of the 777 NFT holders can submit a proposal containing a name, symbol, artwork (IPFS hash), and lore for the next generation
4. **Weighted voting**: 1 NFT = 1 vote. Frens vote on their favorite proposal during a 24-hour voting window
5. **Winner determined**: The proposal with the most votes becomes the next generation's identity
6. **Rebirth executes**: Anyone can call `triggerRebirth()` with the winning proposal's parameters. On-chain verification ensures only the voted-upon identity is accepted

Fallback mechanism: If no proposals receive votes within 72 hours, a deterministic fallback identity is generated from on-chain data (block hash + generation number), ensuring the protocol never gets stuck.

Example: Generation 5 dies during a Bitcoin rally. The frens rally together:

> *Proposal A* by fren #042: "Satoshi's Phoenix" (SPHX) – "From the ashes of generation 5, a bird of orange flame rises, carrying the spirit of Satoshi himself."
> *Proposal B* by fren #777: "Diamond Hands" (DMND) – "They said it was dead. We said we're not selling. Generation 6 is for the holders who never flinched."

Proposal C by fren #369: "Moon Wizard" (MWIZ) – "The wizard looked up. The moon was closer than ever. He began to climb."

The 777 holders vote. The winning identity becomes generation 6. **This is pure community governance**—no algorithms, no oracles, just frens deciding together what their creature becomes next.

Rebirth executes atomically in a single transaction:

1. **Snapshot state**: Emit `GenerationSnapshot` event with current block height
   - All holder balances are retrievable from blockchain history
   - No on-chain storage required (gas efficient)
2. **Remove liquidity**: Burn LP tokens, receive reserves (R_{tokens}, R_{BTC})
3. **Aggregate fees**: Add accumulated protocol fees F_i to R_{BTC}
4. **Deploy generation** $i+1$:
   - New token contract with community-voted metadata
   - Mint exactly R_{tokens} to vault
5. **Create new pool**: Initialize MotoSwap pair for gen $i+1$
6. **Add liquidity**: Deposit $(R_{tokens}, R_{BTC} + F_i)$ to new pool
   - **Critical**: Generation $i+1$ liquidity $>$ generation i liquidity
   - Each rebirth makes the entity more robust
7. **Lock vault**: Transfer LP tokens to vault, locked until next death
8. **Emit events**: `CreatureReborn(i+1, tokenAddress, poolAddress, newLiquidity)`

Liquidity conservation + growth proof: Let L_i^{BTC} be BTC liquidity at generation i. Then:

$$L_{i+1}^{BTC} = L_i^{BTC} + F_i \quad \text{where} \quad F_i = \int_0^{T_i} 0.03 \times vol_{non-NFT}(t)\, dt \tag{6}$$

Since $F_i > 0$ for any generation with trading activity, we have $L_{i+1}^{BTC} > L_i^{BTC}$. Liquidity is strictly increasing.

The unstoppable property: No admin keys exist in the protocol. The vault has no `withdraw()` function except during rebirth. The oracle has no `setThreshold()` function. All parameters are hardcoded in bytecode. Once

deployed, the entity cannot be stopped, paused, or censored. It will rebirth forever.

Metadata storage on-chain:

$$M_i = \{name_i, symbol_i, ipfs_i, lore_i\} \tag{7}$$

where M_i is immutably stored in the `CreatureReborn` event for generation i.

3.6 Phase 6: Holder Claims

Holders from generation i can claim equivalent tokens in generation j (where $j > i$) via `CauldronRegistry.claimFromGeneration(i)`.

Claim verification:

1. Contract retrieves snapshot block height for generation i
2. Queries historical blockchain state at that block
3. Verifies `balanceOf(msg.sender)` at snapshot height
4. Checks $claimed[msg.sender][i] == false$
5. Mints tokens to claimer in current generation
6. Sets $claimed[msg.sender][i] = true$

This is a **pull model**—holders must actively claim. This prevents the protocol from needing to push tokens to thousands of addresses (expensive), while ensuring claims remain available indefinitely (no expiration).

4 The Elder Circle: Loyalty Rewards

4.1 The Problem: Diamond Hands Go Unrewarded

In most token ecosystems, holders who stay through thick and thin receive nothing for their loyalty. The person who buys generation 1 and holds through generation 10 gets the same deal as someone who just arrived. This is unfair.

The Magic Internet Frens Protocol solves this with the **Elder Circle**—a smart contract that tracks consecutive-generation holding streaks and distributes a portion of trading fees proportional to loyalty.

4.2 How the Elder Circle Works

The Elder Circle uses a MasterChef-style O(1) reward accumulator. The core mechanic is simple:

Your reward weight = your streak × your balance.

If you've held through 5 consecutive generations and own 100 tokens, your weight is 500. Someone who just arrived with 100 tokens has a weight of 1. You earn 500× more rewards per token than them.

The check-in system:

1. At the start of each new generation, holders call `checkin()` on the Elder Circle contract
2. The contract checks your last check-in generation:
   - If $lastGen == currentGen - 1$: your streak increments (consecutive!)
   - If $lastGen < currentGen - 1$: your streak resets to 1 (you missed a generation)
3. Your weight is recalculated as $streak \times currentBalance$
4. The global `totalWeight` is updated accordingly

The selling penalty: If you sell any creature tokens, the token contract calls `notifyTransfer()` on the Elder Circle. Your streak is **immediately reset to zero**. All pending rewards are settled first (you don't lose earned rewards), but your future earning power drops to nothing until you check in again.

This creates a powerful incentive: **hold across generations, never sell, and your share of trading fees grows exponentially**.

4.3 Fee Distribution Mathematics

Trading fees flow into the Elder Circle via the `depositFees(amount)` function. The reward accumulator updates as:

$$accRewardPerWeight_{new} = accRewardPerWeight_{old} + \frac{amount \times 10^{18}}{totalWeight} \tag{8}$$

For any holder with weight w_h, pending rewards are:

$$pending_h = \frac{w_h \times accRewardPerWeight}{10^{18}} - rewardDebt_h \tag{9}$$

This is the standard MasterChef formula, proven secure across thousands of DeFi protocols. It allows O(1) reward distribution regardless of the number of holders.

4.4 Streak Multiplier Examples

Holder	Balance	Streak	Weight
Fren A (OG)	100 tokens	10 gens	1,000
Fren B (mid)	100 tokens	3 gens	300
Fren C (new)	100 tokens	1 gen	100
		Total Weight	1,400

Table 2: Elder Circle weight distribution example

If 1 BTC in fees is deposited:

- Fren A receives: $\frac{1000}{1400} \times 1 = 0.714$ BTC (71.4%)
- Fren B receives: $\frac{300}{1400} \times 1 = 0.214$ BTC (21.4%)
- Fren C receives: $\frac{100}{1400} \times 1 = 0.071$ BTC (7.1%)

The OG holder who stayed through 10 generations earns 10× more than the newcomer, even with the same token balance. **Loyalty pays.**

4.5 The Oath and the Broken Oath

The Elder Circle uses thematic naming for its events:

- **OathTaken**: Emitted when a holder checks in. You've sworn loyalty to the circle for another generation.
- **OathBroken**: Emitted when a holder sells. Your streak is reset. You broke the oath.
- **ManaDeposited**: Emitted when trading fees flow into the reward pool. The circle's mana grows.
- **ManaHarvested**: Emitted when a holder claims their accumulated rewards.

This isn't just a staking contract. It's a social contract between frens. Hold together, earn together.

5 Security Analysis

Smart contracts on Bitcoin are immutable. Once deployed, they execute forever with no possibility of upgrade or intervention. This makes security analysis critical—vulnerabilities cannot be patched.

5.1 Protection 1: No Double Claims

The attack: What if someone claims their tokens multiple times from the same generation?

How we prevent it: The contract keeps a permanent record of every claim. Before giving you tokens, it checks: "Has this address already claimed from generation 3?" If yes, transaction fails. If no, you get your tokens and the contract marks you as claimed.

Think of it like a bouncer with a list. Once you're on the list, you can't get in again.

5.2 Protection 2: Liquidity Can't Disappear

The attack: What if liquidity gets lost during rebirth? Or stolen? Or sent to the wrong address?

How we prevent it: All liquidity movement happens in a single atomic transaction. This means either:

- The entire rebirth succeeds (liquidity moves from old pool to new pool), OR
- The entire rebirth fails (everything stays where it was)

There's no in-between state where liquidity is "in transit" and vulnerable. It's all-or-nothing.

The vault acts as a custody system. It receives liquidity from the old pool, then immediately adds it to the new pool. Same transaction. Same block. Instant.

5.3 Protection 3: Volume Can't Be Faked

The attack: What if someone wants to keep a dead token alive by faking trading volume?

How we prevent it: The oracle measures volume over a rolling 24-hour window. You can't just do one big trade to fake life—you need sustained activity.

Cost to attack: Let's say you want to fake enough volume to keep a creature alive when it should be dead. You'd need to trade at least 0.01 BTC per day. Every trade costs you the 3% fee (assuming you don't have the NFT).

So your daily cost is: $0.01 \text{ BTC} \times 3\% \times 144 \text{ blocks} = 0.0432 \text{ BTC/day} \approx \$1,800\text{/day}$

To keep it alive for a month: \$54,000. For a year: \$657,000.

At that point, you're not attacking—you're funding the treasury and paying for the privilege. We'll allow it.

5.4 Protection 4: No Reentrancy Exploits

The attack: Call a function while it's still running to steal funds (like the DAO hack).

How we prevent it: We follow the "Checks-Effects-Interactions" pattern:

1. **Checks**: Verify everything first (is creature dead? does user have balance? has user claimed?)
2. **Effects**: Update the contract state (mark as claimed, update balances)
3. **Interactions**: Only then do external calls (mint tokens, move liquidity)

By updating state before making external calls, there's no exploitable window. The contract knows you claimed before giving you the tokens.

The Elder Circle adds an explicit reentrancy guard: a storage flag that locks during `claimRewards()` and reverts if called recursively.

5.5 Protection 5: NFT Checks Are Real-Time

The attack: Borrow an NFT, trade with 0% fees, return the NFT.

How we prevent it: Every transfer checks your NFT balance at that exact moment. If you don't own the NFT when the transaction executes, you pay 3%. No borrowing exploits. No flash loan tricks.

The contract literally calls the NFT contract and asks: "Does address 0x123... own any NFTs right now?" The answer is always current.

5.6 Protection 6: Elder Circle Streak Integrity

The attack: Game the Elder Circle by transferring tokens between wallets to maintain multiple streaks.

How we prevent it: The token contract notifies the Elder Circle on every outgoing transfer. Any sell resets your streak to zero immediately. You can't "park" tokens in another wallet and claim they're still held—the balance snapshot is refreshed on every check-in by querying the live token contract.

6 The Economics: Why This Actually Makes Sense

6.1 The NFT Value Proposition

There are exactly 777 wizard fren NFTs. No more can ever be minted. If you hold one, you pay 0% trading fees on all creatures, forever.

Should you buy one?

Simple math: If you plan to trade more than 33× the NFT price, you'll save money.

Example:

- NFT costs: 0.1 BTC
- Your trading volume: 10 BTC worth of creatures
- Without NFT: You pay 3% = 0.3 BTC in fees
- With NFT: You pay 0% = 0 BTC in fees
- Your savings: 0.3 BTC − 0.1 BTC = 0.2 BTC profit

The NFT pays for itself. And it works across **all generations forever**—whether that's 10 generations or 1,000 generations. The 0% fee is eternal.

6.2 Why 3% for Non-Holders?

Three reasons:

1. **Sustainable funding**: 3% fees fund the treasury, Elder Circle rewards, and future liquidity
2. **NFT value driver**: The tax creates real utility for NFT holders. Your NFT saves you 3% on every trade.
3. **Not too high, not too low**: High enough to matter. Low enough to not kill trading.

6.3 Fee Distribution Breakdown

The 3% fee collected from non-holder trades is split across three destinations:

- **Liquidity compounding**: A portion flows back into the liquidity pool during rebirth, making each generation more robust
- **Elder Circle rewards**: A portion is deposited into the Elder Circle contract, distributed to loyal holders based on their streak × balance weight
- **Community treasury**: A portion funds community projects, grants, memes, and whatever the frens decide

This creates a triple flywheel: trading generates fees → fees reward holders and grow liquidity → deeper liquidity attracts more trading → more fees. Each component reinforces the others.

6.4 The Treasury Model

The Magic Internet Treasury is a Bitcoin address controlled by the community.

What's it used for?

- Funding new Magic Internet Frens projects
- Grants for builders and artists
- Community events and gatherings
- Memes (the most important category)
- Whatever the frens decide

This isn't a team wallet. It's not VC money. It's community money, from the community, for the community.

7 Technical Implementation

7.1 What It's Built On

- **Blockchain**: Bitcoin Layer 1 via OPNet (not a Layer 2, not a sidechain—actual Bitcoin)
- **Smart Contract Language**: AssemblyScript → compiled to WebAssembly
- **Token Standard**: OP-20 (the OPNet version of ERC-20)
- **NFT Standard**: OP-721 (the OPNet version of ERC-721)
- **DEX**: MotoSwap (Uniswap V2-style AMM built for OPNet)
- **Oracle**: Custom 24-hour TWAP (Time-Weighted Average Price)

7.2 Why Bitcoin?

Because it's the most secure, most decentralized, most censorship-resistant blockchain ever created. If we're building something immortal, it should live on immortal infrastructure.

OPNet enables smart contracts on Bitcoin Layer 1 using Tapscript. No validators. No special tokens. Just Bitcoin.

7.3 Smart Design Choices

Some technical decisions that make this work:

1. **Event-based snapshots**: Instead of storing every holder's balance on-chain (expensive), we emit events and reconstruct state from blockchain history (efficient).

2. **Permissionless rebirth**: Anyone can trigger rebirth when a creature dies. The triggerer provides the community-voted metadata as parameters. On-chain verification ensures the metadata matches the vote outcome.

3. **Community-voted metadata**: The 777 NFT holders propose and vote on each new generation's identity. No centralized oracle, no algorithm—pure community creativity channeled through on-chain governance.

4. **Fully on-chain**: Death detection, liquidity migration, fee distribution, and NFT artwork are all on Bitcoin. No external dependencies that can fail or be censored.

5. **Locked liquidity, unlocked claims**: Liquidity is locked in the vault (can't be rug-pulled), but holder claims are always accessible. You can claim your tokens at any time.

7.4 The Community Voting Architecture

The rebirth voting mechanism operates through a simple on-chain process:

Proposal phase (48 hours after death):

1. Any NFT holder can submit a proposal containing: name (string), symbol (string), IPFS artwork hash (bytes32), and lore (string)

2. Proposals are stored on-chain with the proposer's address

3. Each holder can submit one proposal per voting round

Voting phase (24 hours after proposals close):

1. Each NFT grants 1 vote (non-transferable during voting)

2. Holders vote for their preferred proposal

3. The proposal with the most votes wins

4. In case of a tie, the earlier-submitted proposal wins

Execution phase:

1. Anyone calls `triggerRebirth(proposalId)`
2. Contract verifies: (a) creature is dead, (b) voting period ended, (c) proposal won the vote
3. Rebirth executes atomically with the winning proposal's metadata

Fallback: If no proposals are submitted within 72 hours, a deterministic identity is derived from $hash(blockHash \| generationNumber)$ to ensure the protocol never stalls.

7.5 Open Source Everything

All code is open source and available on GitHub:

- Smart contracts (10 contracts including ElderCircle)
- Frontend components (React + Vite)
- Deployment scripts
- Documentation
- This paper (LaTeX source included)

No secrets. No hidden code. No trust required. Verify everything.

8 Key Numbers

- **777** – Maximum MiFren NFTs (one-time mint, triggers first summoning)
- **49** – Unique trait layers inscribed on Bitcoin
- **183** – Colors in the global on-chain palette
- **5** – Character classes (Wizard, King, Knight, Apprentice, Peasant)
- **11** – Unique face designs shared across all classes
- **0%** – Trading fee for NFT holders (forever)
- **3%** – Trading fee for non-holders
- **0.01 BTC** – Death threshold (24-hour volume)
- **777 tokens + 1 BTC** – Initial liquidity for first creature
- ∞ **generations** – Infinite community-voted evolutions
- **streak** $\times$ **balance** – Elder Circle reward weight formula
- **96 KB** – Total on-chain art storage (98.3% compression)

9 What's Next: The Future of Forever

9.1 V2: Enhanced Community Governance

Right now, the protocol has fixed parameters:

- Death threshold: 0.01 BTC/24h
- Fee rates: 0% / 3%
- Fee split ratios

In the future, the community might vote to adjust these. Or maybe not. Immutability has its charm.

Governance ideas we're considering:

- DAO voting for death threshold adjustments
- Community treasury allocation votes
- Dynamic fee splits based on protocol maturity
- Integration with other Magic Internet Frens projects

But honestly? The protocol works great as-is. Maybe the best governance is no governance—except for choosing what the next creature looks like. That's always up to the frens.

9.2 The Multichain Question

Bitcoin is home. But the Magic Internet Frens concept could work on other chains:

- Ethereum Layer 2s (cheaper fees, same mechanics)
- Other Bitcoin L2s (Lightning Network, RGB Protocol)
- Alt L1s (if we're feeling adventurous)

The core idea—eternal creatures that die and regenerate, governed by their community—is chain-agnostic. But for now, we're Bitcoin-native and proud.

9.3 What the Community Will Build

The Magic Internet Frens Protocol is just infrastructure. The real magic comes from what the frens build on top:

- Trading bots that hunt for arbitrage across creature generations

- Analytics dashboards tracking which creatures live longest
- Games and experiences tied to creature lifecycles
- Art and memes celebrating each creature's unique identity
- Elder Circle leaderboards showing the longest-streaking holders
- Whatever weird ideas emerge from the group chat at 3am

We provide the rails. You provide the train.

10 Conclusion

We have presented the Magic Internet Frens Protocol, the first implementation of an autonomous regenerative token system on Bitcoin Layer 1. The protocol introduces five key innovations:

1. **Autonomous death detection**: On-chain volume oracle enables self-aware lifecycle management without external oracles or admin intervention
2. **Liquidity compounding**: Each generation inherits previous liquidity plus accumulated fees, creating exponential robustness growth over time ($L_{i+1}^{BTC} > L_i^{BTC}$)
3. **Community-driven evolution**: The 777 NFT holders propose and vote on each new generation's identity—name, symbol, artwork, and lore—ensuring every rebirth reflects the collective will of the frens
4. **Elder Circle loyalty rewards**: Holders who maintain their position across consecutive generations earn an increasing share of trading fees, with reward weight growing as $streak \times balance$
5. **True unstoppability**: Zero admin keys, no pause mechanisms, no upgrade paths—once deployed, the entity operates indefinitely with mathematical guarantees

10.1 Comparison to Existing Approaches

Traditional tokens die permanently. Governance-based solutions require active coordination and trusted admins. The Magic Internet Frens Protocol operates autonomously with cryptographic guarantees, rewards loyal holders, and lets the community shape its evolution.

Feature	Traditional	Governance	MIF Protocol
Automatic rebirth	No	No	Yes
Liquidity preservation	No	Manual vote	Atomic
Holder protection	No	If voted	Guaranteed
Loyalty rewards	No	Sometimes	**Elder Circle**
Community-voted identity	No	Rarely	**Every rebirth**
Admin keys	Yes	Yes	**None**
Upgradeable	Yes	Yes	**No**
Pauseable	Yes	Yes	**No**
Growing liquidity	No	No	**Yes**

Table 3: Protocol comparison

10.2 Long-Term Implications

The protocol creates a new category of digital asset: **community-governed autonomous organisms**. Key properties:

- **Indefinite lifespan**: The entity will exist as long as Bitcoin exists
- **Increasing robustness**: $L_{i+1}^{BTC} > L_i^{BTC}$ for all i with trading activity
- **Community ownership**: The 777 frens collectively determine each iteration's identity
- **Loyalty economics**: Long-term holders are systematically rewarded over short-term speculators
- **Zero maintenance**: No human intervention required after deployment
- **Permissionless participation**: Anyone can trade, trigger rebirth, or claim tokens

This represents a fundamental shift from tokens as static assets to tokens as living community-owned organisms.

10.3 The 777 Coordination Problem

The protocol requires 777 participants to mint NFTs before activation. This is not a bug—it's a feature. It ensures:

1. **Sufficient initial interest**: 777 committed participants signal genuine demand
2. **Distributed control**: No single actor can dominate the NFT supply

3. **Aligned incentives**: NFT holders benefit from protocol success (0% fees, voting rights, Elder Circle eligibility)

This is a modern coordination game: 777 actors must unite to summon an entity that then operates independently forever. Once summoned, it cannot be un-summoned.

10.4 Open Questions & Future Work

- Can the mechanism generalize to other asset types (NFTs, stablecoins)?
- What is the optimal death threshold for different market conditions?
- How do multi-generation holders behave (claim immediately vs hold claims)?
- What voting patterns emerge across generations—do frens converge on themes?
- How does the Elder Circle streak distribution evolve over 50+ generations?
- Can cross-chain bridges enable multi-chain regeneration?

10.5 Final Remarks

The Magic Internet Frens Protocol is live code, not a whitepaper promise. All 10 contracts are written, tested, and ready for deployment. Once the 777 NFTs mint, the first generation activates automatically.

What happens then is determined by the community and market forces, not human operators. If trading volume sustains, the entity lives. If volume dies, the frens vote on what comes next, and rebirth triggers. Liquidity compounds. Holders are protected. Loyal frens are rewarded. The cycle continues.

This is unstoppable autonomous infrastructure running on Bitcoin, governed by its community. No admin keys. No pause buttons. No way back.

The code is the law. The frens are eternal.

Acknowledgments

This protocol exists because of:

- **The 777 wizard frens** – Who believed in magic before it was real
- **The OPNet builders** – For making Bitcoin programmable again

- **The MotoSwap team** – For giving creatures a place to trade
- **The Magic Internet Frens community** – For being weird, wonderful, and willing to try impossible things
- **Satoshi Nakamoto** – For the original magic internet money
- **Every degen who ever held a dead token** – You inspired this

We're all gonna make it.

References

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Bitcoin.org.

[2] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum.org.

[3] Adams, H., Zinsmeister, N., & Robinson, D. (2020). *Uniswap v2 Core*. Uniswap Labs.

[4] SushiSwap Team. (2020). *MasterChef: O(1) Reward Distribution for Staking Protocols*. SushiSwap Documentation.

[5] Vogelsteller, F., & Buterin, V. (2015). *ERC-20: Token Standard*. Ethereum Improvement Proposals, no. 20.

[6] Entriken, W., Shirley, D., Evans, J., & Sachs, N. (2018). *ERC-721: Non-Fungible Token Standard*. Ethereum Improvement Proposals, no. 721.

[7] OPNet Development Team. (2024). *OPNet: Smart Contracts on Bitcoin Layer 1*. OPNet.org.

[8] MotoSwap Team. (2024). *MotoSwap: Decentralized Exchange on OPNet*. MotoSwap.finance.

[9] Qin, K., Zhou, L., & Gervais, A. (2021). *Quantifying Blockchain Extractable Value: How dark is the forest?* IEEE S&P.

[10] Wang, Y., Zuest, P., Yao, Y., Lu, Z., & Wattenhofer, R. (2022). *Impact and User Perception of Sandwich Attacks in the DeFi Ecosystem*. ACM CHI.

Contract	Address (Regtest)
CauldronRegistry	TBD
CauldronVault	TBD
VolumeOracle	TBD
MiFrens NFT	TBD
ElderCircle	TBD
CauldronToken Gen 1	TBD
MotoSwap Router	0x80f8375d...02d2ea5a
MotoSwap Factory	0x893f92bb...ea1c883d

Table 4: Deployed Contract Addresses

A Contract Deployment Addresses

B Source Code

Full source code available at:

- Repository: `https://github.com/magic-frens/MagicFrens`
- Smart Contracts: `/contracts/src/`
- NFT Contract: `/contracts/src/nft/MiFrens.ts`
- Elder Circle: `/contracts/src/elder-circle/ElderCircle.ts`
- Frontend: `/src/components/`
- Documentation: `/docs/` (including this paper's LaTeX source)
- Deployment Scripts: `/contracts/scripts/`

Key files:

- `MiFrens.ts` – 777 on-chain wizard NFTs with SVG rendering (1,600+ lines)
- `CauldronToken.ts` – The eternal creature token
- `CauldronRegistry.ts` – Rebirth orchestrator
- `ElderCircle.ts` – Loyalty streak rewards (500+ lines)
- `VolumeOracle.ts` – Death detection
- `CauldronVault.ts` – Liquidity custodian

Everything is MIT licensed. Fork it. Study it. Improve it. Make your own magic.